

TP5 - Sistemas Operativos

1. ¿Se puede producir deadlock sin estar involucrados recursos del sistema? En caso de ser cierto de un ejemplo.

Sí que podría darse, ya que si se cumplen las 4 condiciones que son exclusión mutua, retener y esperar, no apropiación y espera circular, no son únicas a recursos del sistema, sino que recursos en general, hasta de la vida real.

- Dos autos cruzando un puente en una sola dirección desde direcciones opuestas.
- Una persona bajando una escalera mientras otra persona sube por ella.
- Dos trenes viajando cada uno hacia el otro en el mismo raíl.
- Dos impresoras pueden bloquearse si cada una de las otras, esperan a que liberen el dispositivo.

A su vez también hay ejemplos un poco más asociados a la materia. Un hilo puede usar recursos de otro proceso, si se da una comunicación de interprocesos (las cuales pueden ser a través de cola de mensajes, o memoria compartida).

- Pipes sin buffer: si dos procesos se comunican en esta situación, y cada uno espera que el otro escriba algo antes de leer, pueden bloquearse mutuamente.
- Mensaje sin acuse de recibo: en sistemas de mensaje, si un proceso envía un mensaje y espera una respuesta, pero el otro proceso no envía la respuesta, puede ocurrir deadlock.

2. Suponga que un sistema se encuentra en estado inseguro. Muestre que es posible que los procesos completen su ejecución sin entrar en un estado de interbloqueo.

Antes que nada habría que explicar cuando un sistema se encuentra en un estado inseguro. Cuando un proceso requiere un recurso disponible, hay que ver si se le otorga ese recurso deja al sistema en un estado inseguro. Estará en estado seguro si existe una secuencia segura de ejecución de todos los procesos.

Supongamos que tenemos 3 procesos y 12 recursos. P1 necesita 4 y tiene 2, P2 necesita 9 y tiene 2, y P3 necesita 10 y tiene 5. Eso nos deja con 3 recursos libres. Supongamos que P2 pide 3 recursos, y se los da. Eso nos deja en un estado inseguro (ya que puede ser que ocurra un deadlock), sin embargo, si a partir de ahí P2 libera esos dos recursos, se está de vuelta en un estado seguro, y ahí podría completarse la ejecución.

3. Una computadora tiene seis unidades de cinta, con n procesos en competencia por ellas. Cada proceso necesita dos unidades. ¿Para cuáles valores de n se libera el bloqueo del sistema?

28. A computer has X tape drives, with n processes competing for them. Each process may need up to m drives. How many tape drives should the system have to guarantee that it is deadlock free?

28. Suppose that process A requests the records in the order a, b, c . If process B also asks for a first, one of them will get it and the other will block. This situation is always deadlock free since the winner can now run to completion without interference. Of the four other combinations, some may lead to deadlock and some are deadlock free. The six cases are as follows:

a b c	deadlock free
a c b	deadlock free
b a c	possible deadlock
b c a	possible deadlock
c a b	possible deadlock
c b a	possible deadlock

Since four of the six may lead to deadlock, there is a $1/3$ chance of avoiding a deadlock and a $2/3$ chance of getting one.

4. ¿Puede ocurrir deadlock en la siguiente situación? Explique

Proceso P	Proceso Q
...	...
Requerir A	Requerir B
...	...
Liberar A	Requerir A
...	...
Requerir B	Liberar B
...	...
Liberar B	Liberar A

No, no puede ocurrir deadlock para la siguiente situación. Esto se debe a que no cumple con una de las condiciones de los deadlock, y es que nunca se forma un ciclo y no hay espera circular. Esto se debe a que se hacen las cosas paso por paso, y no piden los recursos P y Q en cualquier momento, sino después de asegurarse de que el otro lo haya liberado.

5. ¿Cómo utilizaría un algoritmo de detección de deadlock (en que momento se ejecutaría)?
¿Qué haría el algoritmo si detecta que hay deadlock?

Lo utilizaría primero que todo invocandolo periódicamente, donde el periodo dependa de qué tan probable es que ocurra un deadlock. Si es bastante probable, entonces sería conveniente hacerlo lo antes posible, ya que al retrasarse puede provocar que el número de programas en deadlock aumente.

Puede chequearse o bien después de otorgar un recurso caro, o mediante métricas de uso del CPU, ya que en muchos casos los deadlock tienden a bajar el uso de la misma.

Si se llega a detectar el deadlock , tenemos varias opciones:

- terminar todos los procesos que están en deadlock: la desventaja es que puede llegar a ser muy caro, ya que todo lo que estaban haciendo se va a perder.
- terminar los procesos de a uno viendo si se resolvió el deadlock: puede llegar a ser caro también, pero minimizado. Lo que sí hay ahora es un costo adicional que es el hacer el algoritmo muchas veces.

Generalmente esto es peligroso, porque podrían estar ambos métodos por ejemplo editando un archivo, o modificando parámetros de algo de tiempo real duro, y estos quedarían en estados corruptos. En el caso de ir terminando de a uno, se podría eliminar el que conlleva el menor costo (se basa en varias cosas esa decisión, por ejemplo, menor tiempo de ejecución, menor prioridad, cuantos recursos usa, cuantos usará, etc).

También está la recuperación de Interbloqueos mediante apropiación de recursos, donde se podría elegir un proceso y quitarle un recurso para dárselo a otro, es decir si está reteniendo y esperando le sacamos esa condición. El problema es que habría que ver algunas cuestiones:

- Selecciona una víctima: tiene que ser el menor costo.
- Rollback: una vez que se elige la víctima, al no tener ese recurso hay que volver a esta a un estado seguro.
- Inanición: para evitar que el mismo proceso sea la víctima, hay que hacer que se tenga eso en cuenta para la siguiente elección. Por ej: teniendo en cuenta el número de rollbacks en el menor costo.

6. Discuta hasta qué punto son útiles los algoritmos de prevención y de detección de bloqueos.

Detectar interbloqueos y usar estos algoritmos generalmente tienen un costo altísimo. No es por nada que muchos sistemas operativos como Linux, Windows, deciden optar por ignorar los interbloqueos por su costo y también baja probabilidad. Entonces, si serían útiles en casos de alta prioridad de deadlocks, y mientras sea necesario evitar que ocurra ya que tampoco queremos que se nos viva bloqueando el sistema. Para ponernos en onda, el algoritmo del banquero para saber secuencias de estados seguros, tiene en el peor caso un orden de $O(m \times n^2)$, sumado a las muchas estructuras de datos que se usan. Por lo que retomando lo de antes, en SO actuales no tendría mucho sentido, ya que sólo sí se son para en Windows por ejemplo lo reiniciamos y ya está, que serán pocas veces.

Aclaración: El algoritmo de asignación de recursos con grafos funciona mientras no haya varias instancias de un recurso, en ese caso se usa el del banquero.

An argument for installing deadlock avoidance in the system is that we could ensure that deadlock would never occur. In addition, despite the increase in turnaround time, all 5,000 jobs could still run. An argument against installing deadlock-avoidance software is that deadlocks occur infrequently, and they cost little when they do occur.

7. Considere un sistema que ejecuta 5.000 trabajos por mes y no tiene un esquema de prevención o evasión de interbloqueos. Los interbloqueos se producen aproximadamente dos veces al mes, y el operador debe finalizar y volver a ejecutar unos diez trabajos en cada uno. Cada trabajo cuesta aproximadamente dos dólares (en tiempo de CPU), y los trabajos cancelados tienden a estar a la mitad de su tiempo de procesamiento.

Proceso	Asignado	Máximo	Disponible
P0	0 0 1 2	0 0 1 2	1 5 2 0
P1	1 0 0 0	1 7 5 0	
P2	1 3 5 4	2 3 5 6	
P3	0 6 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

Figura 1: Estado del Sistema

El jefe de sistemas ha estimado que un algoritmo de evasión de interbloqueos (como el algoritmo del banco) podría instalarse en el sistema con un aumento de aproximadamente 10 por ciento en el tiempo promedio de ejecución por trabajo. Dado que la máquina actualmente tiene un 30 % de tiempo de inactividad, los 5,000 trabajos por mes aún podrían ejecutarse, aunque el tiempo de respuesta aumentaría en un 20 % en promedio.

a) ¿Cuáles son los argumentos para instalar el algoritmo de evasión?

b) ¿Cuáles son los argumentos contra la instalación del algoritmo de evasión?

8. De tres maneras de eliminar un bloqueo.

Las mismas que antes, rollback, seleccionar una víctima y la inanición.

9. Defina que es un estado seguro de un sistema

Un estado seguro del sistema se da si existe una secuencia segura de ejecución de todos los procesos. Esto es, dada esa secuencia en la que manejamos los recursos del sistema de forma tal de que no haya posibilidad de que se genere un deadlock.

10. ¿Qué diferencia hay entre deadlock y starvation?

En un deadlock, un proceso P1 no puede avanzar debido a que necesita un recurso que lo tiene otro proceso P2, y P2 no puede avanzar debido a que necesita un recurso que tiene P1. Es decir ocurre un problema que cada uno afecta al otro.

Por el lado de la inanición, un proceso nunca puede acceder a un recurso debido a que nunca puede asegurarse. Esto por que hay procesos de mayor prioridad que siempre lo agarran, u otro proceso de la misma prioridad llega a agarrarlo antes que él. Además, un deadlock siempre es entre dos procesos o más, mientras que con la inanición si está la posibilidad de que le ocurra a un solo proceso.

11. Considere la situación del sistema que aparece en la figura 1.

Proceso	Asignado	Máximo	Disponible
P0	0 0 1 2	0 0 1 2	1 5 2 0
P1	1 0 0 0	1 7 5 0	
P2	1 3 5 4	2 3 5 6	
P3	0 6 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

a) Escriba la tabla de necesidad

Hecho en hoja

b) ¿Se encuentra en estado seguro?

Sí para una de las secuencia encontradas: < P0, P2, P3, P4, P1 >

c) Supongamos que el proceso P1 solicita (0, 4, 2, 0). ¿Puede satisfacerse de inmediato?

12. ¿Cuáles son las dificultades que se presentan cuando un proceso es 'rolled-back' ? Comente también sus ventajas si las hubiere.

Una dificultad es que, al hacer el rollback, nada impide que el deadlock original ocurra de vuelta. Además, ¿hasta qué punto se hace el rollback? Normalmente, se pone un checkpoint, y se puede hacer el rollback hasta el checkpoint que sea necesario. Esto a su vez tiene una carga sobre el sistema, ya que es necesario ir manteniendo información sobre los checkpoints para cada proceso. Aparte, ciertos cálculos que el proceso pudo haber realizado hasta ese punto serán perdidos.

Si bien no se ve ninguna ventaja a simple vista, el hecho de usar el rollback en sí permite evitar, por ejemplo, terminar procesos cuando no era necesario, lo que evita aún mayores pérdidas de datos.

Cuando un proceso se revierte como resultado de un bloqueo, la dificultad que surge es la inanición, ya que cuando revertimos el proceso, puede existir la posibilidad de que vuelva a ocurrir un bloqueo, después de lo cual el proceso esperará nuevamente por una cantidad de tiempo indefinida, por lo que se queda sin recursos. Por otro lado, la baja utilización del dispositivo y el robo de ciclos no son conceptos relacionados con la reversión del proceso como resultado de un bloqueo.

13. Dado un sistema controlado por el algoritmo del 'Banquero' (banker). ¿Cuáles de los siguientes cambios pueden ser realizados sin pasar a un estado 'inseguro' (unsafe)?

a) Incremento Disponible (nuevos recursos).

Puede ser realizado, y de hecho es una mejora para el sistema.

b) Decremento Disponible.

No, ya que puede ser que el safe state anterior dependiera de esos recursos que son sacados.

c) Incremento MAX.

No podría hacerse, ya que estarías pidiendo aún más recursos lo que si estabas justo antes con algunos te podría dejar en estado inseguro.

d) Decremento MAX.

Sí que se podría hacer. Pedir menos recursos.

e) Incremento del Número de Procesos.

Podría hacerse, mientras no pidan esos recursos. Sin embargo podría llegar un proceso que pida 12 recursos por ejemplo, y ni devolviendo todos los que tienen los otros recursos luego de una secuencia segura podrían obtenerse.

f) Decremento del Número de Procesos.

Sí, para incrementar el uso de recursos también.